FICHES DE RÉALISATION PROFESSIONNELLES

Test de connection :

Maintenant, on se connecte à la box en SSH.
On vérifie le numéro des modems ; ici, on voit deux modems activés. L'un est connecté à mon ordinateur et l'autre à une carte SIM.

```
root@steer-f85e3c6bac00:~# mmcli -L
    /org/freedesktop/ModemManager1/Modem/13 [Quectel] RM520N-GL
    /org/freedesktop/ModemManager1/Modem/14 [Quectel] RM520N-GL
```

Ensuite, on peut récupérer les infos sur le modem que l'on veut. Ici, c'est celui qui est connecté sur la carte SIM (on utilise le chiffre que l'on retrouve à la fin).

```
root@steer-f85e3c6bac00:~# mmcli -m 13
```

En premier, on a les informations générales du modem sur lequel la carte SIM est connectée.

```
  -----------------------------------
  General   |                   path: /org/freedesktop/ModemManager1/Modem/13
            |              device id: ca625a6fb781670180d7d4617d3d8d1dd506c612
```

Il y a le chemin du modem et l'ID de l'appareil.

Les composants du modem :

```
  -----------------------------------
  Hardware  |            manufacturer: Quectel
            |                   model: RM520N-GL
            |       firmware revision: RM520NGLAAR03A03M4G_BETA_20240522C
            |          carrier config: ROW_Commercial
            | carrier config revision: 0A010809
            |            h/w revision: RM520N-GL
            |               supported: gsm-umts, lte, 5gnr
            |                 current: gsm-umts, lte, 5gnr
            |            equipment id: 868371053173882
```

Le fabricant Quectel est le fabricant et fournisseur des modules de communication cellulaire. Il y a aussi le modèle du module, puis la version du micrologiciel utilisé. "Carrier config" signifie qu'il est destiné à être utilisé dans la région "rest of world", en dehors des régions spécifiques comme l'Amérique du Nord ou l'Europe. "Carrier config revision" est la chaîne qui représente la révision spécifique de la configuration de l'opérateur. Les révisions permettent de faire des mises à jour. La ligne "h/w révision" nous montre que c'est un module 5G de 6 GHz. On peut voir sur les lignes "supported" et "current" les réseaux qu'il peut utiliser et ceux couramment utilisés. C'est un réseau cellulaire grâce à GSM-UMTS, et il peut se connecter de la 3G à la 5G grâce à "LTE" et "5GNR". La dernière ligne est l'ID de l'équipement.

Le système :

```
----------------------------------
System    |                 device: /sys/devices/platform/soc/11200000.usb/usb2/2-1/2-1.1
          |                physdev: /sys/devices/platform/soc/11200000.usb/usb2/2-1/2-1.1
          |                drivers: option1, cdc_mbim
          |                 plugin: quectel
          |           primary port: cdc-wdm0
          |                  ports: cdc-wdm0 (mbim), ttyUSB0 (ignored), ttyUSB1 (gps),
          |                         wwan0 (net)
```

Les deux premières lignes sont les chemins entiers pour pouvoir accéder aux mêmes informations. Grâce à ce chemin, on voit que c'est le premier modem sur lequel on est connecté.

Le statut du modem :

```
------------------------------------
Status    |                    lock: sim-pin2
          |          unlock retries: sim-pin2 (3)
          |                   state: connected
          |             power state: on
          |             access tech: lte, 5gnr
          |          signal quality: 0% (cached)
```

Le modem est bien connecté et allumé. Il a accès de la 3G à la 5G.

Les modes de connexion :

```
----------------------------------
Modes     |               supported: allowed: 3g; preferred: none
          |                          allowed: 4g; preferred: none
          |                          allowed: 3g, 4g; preferred: 4g
          |                          allowed: 3g, 4g; preferred: 3g
          |                          allowed: 5g; preferred: none
          |                          allowed: 4g, 5g; preferred: 5g
          |                          allowed: 4g, 5g; preferred: 4g
          |                          allowed: 3g, 5g; preferred: 5g
          |                          allowed: 3g, 5g; preferred: 3g
          |                          allowed: 3g, 4g, 5g; preferred: 5g
          |                          allowed: 3g, 4g, 5g; preferred: 4g
          |                          allowed: 3g, 4g, 5g; preferred: 3g
          |                  current: allowed: 3g, 4g, 5g; preferred: 5g
```

Cela nous montre les modes de connexion supportés et les préférés, ainsi que ceux couramment utilisés.

Les bandes réseau :

```
----------------------------------
Bands     |              supported: utran-1, utran-4, utran-6, utran-5, utran-8, utran-2,
          |                         eutran-1, eutran-2, eutran-3, eutran-4, eutran-5, eutran-7, eutran-8,
          |                         eutran-12, eutran-13, eutran-14, eutran-17, eutran-18, eutran-19,
          |                         eutran-20, eutran-25, eutran-26, eutran-28, eutran-29, eutran-30,
          |                         eutran-32, eutran-34, eutran-38, eutran-39, eutran-40, eutran-41,
          |                         eutran-42, eutran-43, eutran-46, eutran-48, eutran-66, eutran-71,
          |                         utran-19, ngran-1, ngran-2, ngran-3, ngran-5, ngran-7, ngran-8,
          |                         ngran-12, ngran-13, ngran-14, ngran-18, ngran-20, ngran-25, ngran-26,
          |                         ngran-28, ngran-29, ngran-30, ngran-38, ngran-40, ngran-41, ngran-48,
          |                         ngran-66, ngran-70, ngran-71, ngran-75, ngran-76, ngran-77, ngran-78,
          |                         ngran-79
          |                 current: utran-1, utran-4, utran-5, utran-8, utran-2, eutran-1,
          |                         eutran-2, eutran-3, eutran-4, eutran-5, eutran-7, eutran-8,
          |                         eutran-12, eutran-13, eutran-14, eutran-17, eutran-18, eutran-19,
          |                         eutran-20, eutran-25, eutran-26, eutran-28, eutran-29, eutran-30,
          |                         eutran-32, eutran-34, eutran-38, eutran-39, eutran-40, eutran-41,
          |                         eutran-42, eutran-43, eutran-46, eutran-48, eutran-66, eutran-71,
          |                         utran-19, ngran-1, ngran-2, ngran-3, ngran-5, ngran-7, ngran-8,
          |                         ngran-12, ngran-13, ngran-14, ngran-20, ngran-25, ngran-26, ngran-28,
          |                         ngran-29, ngran-30, ngran-38, ngran-40, ngran-41, ngran-48, ngran-66,
          |                         ngran-71, ngran-75, ngran-76, ngran-77, ngran-78, ngran-79
```

L'UTRAN est la bande utilisée pour les réseaux 3G, l'EUTRAN pour les réseaux 4G et l'NGRAN pour les réseaux 5G.

```
----------------------------------
IP        |              supported: ipv4, ipv6, ipv4v6
```

Les protocoles utilisés.

Les informations sur la carte SIM connectée :

```
----------------------------------
3GPP      |                    imei: 868371053173882
          |           enabled locks: fixed-dialing
          |             operator id: 20801
          |           operator name: Orange F
          |            registration: roaming
          |    packet service state: attached
          |                     pco: 0: (partial) '270480000000'
```

L'IMEI est un numéro unique utilisé pour identifier les appareils mobiles. "Enable locks" permet de restreindre l'utilisation de l'appareil à un ensemble prédéfini de numéros de téléphone. L'ID est l'identifiant pour identifier l'opérateur qui est Orange France. La "registration" nous montre que le réseau n'est pas celui de l'opérateur d'origine. "Packet service state" signifie que votre appareil est connecté au réseau de données mobiles et peut envoyer et recevoir des données. Le "PCO" est la configuration des paramètres spécifiques pour la transmission de données sur le réseau mobile.

```
----------------------------------
3GPP EPS  |    ue mode of operation: csps-2
          |     initial bearer path: /org/freedesktop/ModemManager1/Bearer/0
          |  initial bearer ip type: ipv4v6
```

Le 3GPP est un organisme qui développe des protocoles pour les télécommunications mobiles. L'EPS est une architecture de réseau définie par le 3GPP pour les réseaux mobiles de quatrième génération (4G). Cela concerne le mode de fonctionnement et les paramètres initiaux de l'appareil mobile. "UE" signifie "User Equipment", "CSPS" signifie "Control and Service Plane Séparation". L'initial bearer path est le système Linux pour gérer les modems et les connexions de données mobiles et prend en charge l'IPv4 et l'IPv6.

```
  ------------------------------------
  3GPP 5GNR |                mico mode: unsupported
            |                drx cycle: unsupported
```

5GNR est la norme radio pour la 5G, configurations spécifiques pour des petites cellules qui sont utilisées pour améliorer la couverture et la capacité dans des zones densément peuplées. DRX est une technique utilisée pour économiser la batterie des appareils mobiles.

Les informations sur la carte SIM qui sont dans les ports :

```
  ------------------------------------
  SIM       |        primary sim path: /org/freedesktop/ModemManager1/SIM/0
            |          sim slot paths: slot 1: /org/freedesktop/ModemManager1/SIM/0 (active)
            |                          slot 2: none
```

"Primary SIM path" indique le chemin qui montre l'emplacement de la carte SIM principale gérée par ModemManager. "SIM slot paths" indique que le slot 1 est occupé par une carte SIM et que le slot 2 est vide. Le chemin fait référence à un bearer qui est essentiellement une connexion de données mobile active.

```
  ------------------------------------
  Bearer    |                   paths: /org/freedesktop/ModemManager1/Bearer/1
```

référence à un bearer qui est essentiellement une connexion de données mobile active.

Script de test de connexion :

```
ROUGE='\033[0;31m'
ORANGE='\e[38;5;208m'
JAUNE='\e[33m'
VERT='\033[0;32m'
BLEU='\033[0;34m'
NC='\033[0m'
```

Initialise les codes couleur pour pouvoir les utiliser ensuite afin qu'ils s'affichent dans le terminal.

```
echo "
```

Ensuite, il y a une petite décoration en ASCII qui s'affiche tout le temps car elle n'est pas dans une fonction.

```
#--------------- MAIN CODE -------------------
case "$1" in
    ping)
        ping_test
        ;;
    dns)
        dns_test
        ;;
    signal | s)
        signal_quality
        ;;
    ""|all)
        ping_test
        dns_test
        signal_quality
        ;;

esac
```

Cette partie du script permet d'utiliser les fonctions indépendamment des autres. Tout d'abord, il analyse le premier argument passé au script ($1), qui permet à l'utilisateur qui veut que le ping soit effectué. C'est pareil pour le DNS et le signal (pour le signal on peut juste mettre un “s” aussi) , et le all exécute toutes les fonctions.

```
ping_test() {
    echo -e "-------------------------- Ping test ----------------------------"
    if ping -c 1 "8.8.8.8" > /dev/null 2>&1; then
        echo -e "${VERT}successful ping.${NC}"
    else
        echo -e "${ROUGE}ping failure.${NC}"
    fi
    echo -e "-----------------------------------------------------------------"
}
```

Cette partie du script permet de réaliser un test de ping :
Le script effectue un seul envoi de paquet ICMP à l'adresse IP 8.8.8.8 et n'affiche pas le ping grâce à sa commande "> /dev/null" qui permet de masquer la réponse. "if [? -eq 0]; then" vérifie si la commande a réussi. Ensuite, si le ping réussit, un message "Ping réussi" s'affiche, sinon il envoie "Échec du ping" en rouge et "Ping réussi" s'affiche en couleur verte. Le "\{NC}" permet d'enlever la couleur pour la ligne suivante. Les deux "echo" au début et à la fin permettent de décorer un peu le résultat. Il est dans une fonction pour pouvoir réaliser seulement cette partie du script.

Voici ce que ça donne si le ping marche.

Et voici ce qu'il affiche quand il ne fonctionne pas

```
dns_test() {
    echo -e "-------------------------- Test DNS ------------------------------"
    result=$(dig +short google.com)

    if [ -n "$result" ]; then
        echo -e "${VERT}DNS test successful.${NC}"
    else
        echo -e "${ROUGE}DNS test failure.${NC}"
    fi
}
```

Celle-là permet seulement les tests DNS :
La commande "dig" est utilisée pour effectuer une requête DNS et il affiche "Test DNS réussi" en vert si ça marche et "Échec du test DNS" en rouge s'il ne fonctionne pas.

Voici quand le test DNS marche.

et voici quand il ne marche pas

Il échoue quand on change le "google.com" par n'importe quoi comme "rejvdfkvbj".

```
signal_quality() {
    echo -e "------------------------- Signal Quality --------------------------"

echo -n "Choose a modem (1/2): "
read -r modem

MODEM1="/sys/devices/platform/soc/11200000.usb/usb1/1-1/1-1.1"
MODEM2="/sys/devices/platform/soc/11200000.usb/usb1/1-1/1-1.2"

if [[ "$modem" == "1" ]]; then
    response=$(mmcli -m "$MODEM1" --command 'AT+QENG="servingcell"' 2>&1)
    if echo "$response" | grep -q "couldn't find modem"; then
        echo -e "${ROUGE}Modem 1 not found${NC}"
        exit 1
    fi
    echo -e "${BLEU}Modem 1 is used${NC}"
    operator_response=$(mmcli -m "$MODEM1" --command 'AT+COPS?' 2>&1)
elif [[ "$modem" == "2" ]]; then
    response=$(mmcli -m "$MODEM2" --command 'AT+QENG="servingcell"' 2>&1)
    if echo "$response" | grep -q "couldn't find modem"; then
        echo -e "${ROUGE}Modem 2 not found${NC}"
        exit 1
    fi
    echo -e "${BLEU}Modem 2 is used${NC}"
    operator_response=$(mmcli -m "$MODEM2" --command 'AT+COPS?' 2>&1)
else
    echo "Invalid choice"
    exit 1
fi
```

Cette partie est dans la fonction “`signal_quality`”. La ligne qui suit l'initialisation de la fonction affiche un en-tête. Ensuite, le script demande à l'utilisateur s'il veut faire ce test sur le modem 1 ou 2, puis il initialise le chemin des deux modems. Le script utilise “`mmcli`” et envoie une commande AT au premier ou au deuxième modem selon le choix de l'utilisateur. S'il répond, alors le script affiche “`Modem 1 is used`” sinon il affiche “`Modem 1 not found`”, et pareil pour le modem 2. Il permet également de pouvoir afficher l’opérateur utilisé

```
operator=$(echo "$operator_response" | sed -n 's/.*"\(.*\)".*/\1/p')
echo -e "${BLEU}Operator:${NC} $operator${NC}"
```

Donne le nom de l’opérateur du modem utilisé

```
rsrp=$(echo "$response" | awk -F',' '{print $14}' | tr -d '[:space:]')
rsrq=$(echo "$response" | awk -F',' '{print $15}' | tr -d '[:space:]')
rssi=$(echo "$response" | awk -F',' '{print $16}' | tr -d '[:space:]')
sinr=$(echo "$response" | awk -F',' '{print $18}' | tr -d '[:space:]')
```

Cela initialise "rsrp", "rsrq", "rssi" et "sinr".
Quand un modem est trouvé, il exécute donc la commande "mmcli -m (le modem trouvé) –command" et associe les valeurs 14, 15, 16 et 18 à leurs variables. Par exemple, si la réponse est
"+QENG:
"servingcell","NOCONN","LTE","FDD",208,20,123456,400,100,3,10,-95,-10,-75,30,100,200",
le script récupère les valeurs demandées et les associe à leurs variables respectives et supprime les espaces qui peuvent fausser les résultats ou même le faire échouer .

```
    if (( rsrp >= -90 )); then
        echo -e "${VERT}RSRP: Excellent${NC}"
    elif (( rsrp >= -105 && rsrp < -90 )); then
        echo -e "${JAUNE}RSRP: Good${NC}"
    elif (( rsrp >= -115 && rsrp < -105 )); then
        echo -e "${ORANGE}RSRP: Fair${NC}"
    else
        echo -e "${ROUGE}RSRP: Poor${NC}"
    fi

    if (( rsrq >= -10 )); then
        echo -e "${VERT}RSRQ: Excellent${NC}"
    elif (( rsrq >= -15 && rsrq < -10 )); then
        echo -e "${JAUNE}RSRQ: Good${NC}"
    elif (( rsrq >= -20 && rsrq < -15 )); then
        echo -e "${ORANGE}RSRQ: Fair${NC}"
    else
        echo -e "${ROUGE}RSRQ: Poor${NC}"
    fi

    if (( rssi >= -70 )); then
        echo -e "${VERT}RSSI: Excellent${NC}"
    elif (( rssi >= -85 && rssi < -70 )); then
        echo -e "${JAUNE}RSSI: Good${NC}"
    elif (( rssi >= -100 && rssi < -85 )); then
        echo -e "${ORANGE}RSSI: Fair${NC}"
    else
        echo -e "${ROUGE}RSSI: Poor${NC}"
    fi

    if [[ -z "$sinr" || "$sinr" == "-" ]]; then
    sinr=0
    fi
    if (( sinr >= 20 )); then
        echo -e "${VERT}SINR: Excellent${NC}"
    elif (( sinr >= 13 && sinr < 20 )); then
        echo -e "${JAUNE}SINR: Good${NC}"
    elif (( sinr > 0 && sinr < 13 )); then
        echo -e "${ORANGE}SINR: Fair${NC}"
    elif (( sinr == 0 )); then
        echo -e "${ROUGE}SINR: Poor${NC}"
    else
        echo -e "${ROUGE}SINR: Null${NC}"
    fi

}
```

Ensuite, les valeurs obtenues sont analysées. Par exemple, si "rsrp" est au-dessus de -90, cela renvoie "RSRP : Excellent". S'il est entre -105 et -90, cela renvoie "RSRP: Good". S'il est entre -115 et -105, cela renvoie "RSRP: Fair". Et s'il est en dessous de -115, cela renvoie "RSRQ: Poor". Pour le dernier, il y a 2 lignes en plus car le SINR peut être parfois égal à 0, ce qui permet de le forcer au numéro 0.

Voici quand ça marche.

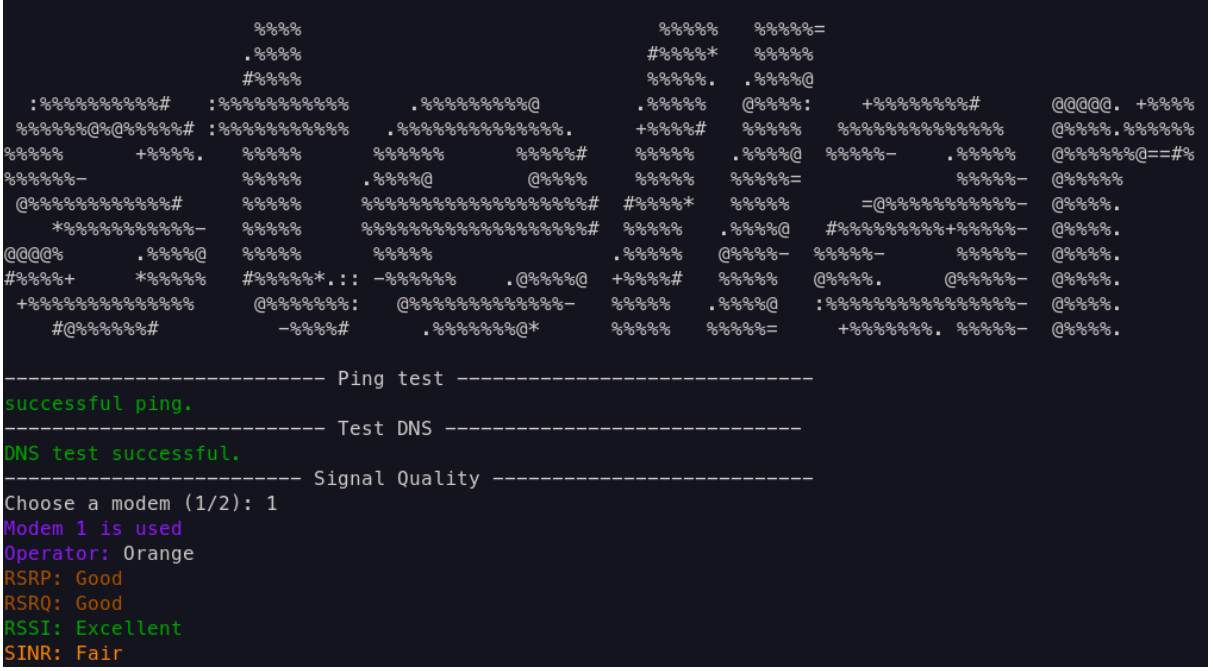
```
root@steer-f85e3c46e2a8:~# /tmp/pouet.sh s

------------------------- Signal Quality ---------------------------
Choose a modem (1/2): 1
Modem 1 is used
Operator: Orange
RSRP: Good
RSRQ: Good
RSSI: Excellent
SINR: Excellent
```

Voici quand ca ne marche pas :

```
--------------------------- Ping test ------------------------------
successful ping.
--------------------------- Test DNS -------------------------------
DNS test successful.
------------------------- Signal Quality ---------------------------
Choose a modem (1/2): 1
Modem 1 not found
```

Voici ce s’affiche quand tout le script est exécuté en même temps :

```
root@steer-f85e3c46e2a8:~# /tmp/pouet.sh
```



```
--------------------------- Ping test ------------------------------
successful ping.
--------------------------- Test DNS -------------------------------
DNS test successful.
------------------------- Signal Quality ---------------------------
Choose a modem (1/2): 1
Modem 1 is used
Operator: Orange
RSRP: Good
RSRQ: Good
RSSI: Excellent
SINR: Fair
```

Test de failover :

Un failover permet de basculer d'un réseau à un autre en cas de panne du réseau principal. Par exemple, si le réseau principal est satellitaire et qu'il y a une panne, cela permet à la box de passer directement à un réseau différent, comme une connexion cellulaire via un opérateur réseau tel que Free ou Orange.

Le test permet de vérifier si la connexion bascule bien d'un réseau à l'autre et permet de voir les pertes de paquets engendrées pendant le basculement. Donc, pour réaliser ce test, on a besoin de deux réseaux. Pour mon cas, c'est le réseau satellite Starlink et un réseau cellulaire d'Orange.

```
------------------------------------
3GPP      |                   imei: 868371053173882
          |           enabled locks: fixed-dialing
          |             operator id: 20801
          |           operator name: Orange F
          |            registration: roaming
          |    packet service state: attached
          |                    pco: 0: (partial) '270480000000'
```

Donc, pour commencer, on vérifie si le réseau principal est bien connecté avec la commande :
logread -f | grep failover

Cette commande permet d'afficher les logs. Le "-f" permet d'avoir les mises à jour des logs sans devoir refaire la commande. "|" et "grep failover" affichent seulement les messages contenant le mot "failover".

```
melih@SARAY:~$ logread -f | grep failover

2025-06-12T08:08:43+00:00 steer-f85e3c6458b8 failover: Default route set to eth1 via 100.64.0.1
2025-06-12T08:08:59+00:00 steer-f85e3c6458b8 failover: Default route set to wwan1 via 10.115.100.94
2025-06-12T08:10:27+00:00 steer-f85e3c6458b8 failover: Default route set to eth1 via 100.64.0.1
2025-06-12T08:11:11+00:00 steer-f85e3c6458b8 failover: Default route set to wwan1 via 10.115.100.94
2025-06-12T08:11:27+00:00 steer-f85e3c6458b8 failover: Default route set to eth1 via 100.64.0.1
2025-06-12T08:11:42+00:00 steer-f85e3c6458b8 failover: Default route set to wwan1 via 10.115.100.94
```

Sur la capture, on voit que quand on débranche le réseau principal qui est eth1, il passe à wwan1. Il faut faire un ping en même temps que le basculement.

```
--- 1.1.1.1 ping statistics ---
50 packets transmitted, 49 received, 2% packet loss, time 49090ms
rtt min/avg/max/mdev = 20.209/58.610/181.425/30.936 ms
```

Pour réussir le test, on lance d'abord le ping avec le réseau principal, puis à la moitié, donc ici à 25, on débranche le réseau pour simuler une panne et il y a un basculement vers le réseau cellulaire. Ici, on remarque une petite perte de 2 % de paquets au moment du basculement, ce qui est un bon résultat.

Ensuite, il y a une étape avancée où, si le réseau satellite et le réseau cellulaire de la première carte SIM ne marchent pas, la box se connecte à la deuxième carte SIM